CENG222- HOMEWORK 3

Ekin

Contents

Part 1:

Bisection Method:

How it Works:

The Binary Search algorithm approaches the problem as a root-finding challenge within a bounded, sorted range. Since the integer square root of a non-negative integer **N** must lie somewhere between **0** and **N**, we can set our initial search space with a lower bound (**a** = **0**) and an upper bound (**b** = **N**).

In each iteration, the algorithm finds the middle point (**c = (a + b) / 2**) and checks its square. To prevent integer overflow during assembly implementation, instead of evaluating **c x c <= N**, we can mathematically rewrite it as **c <= N/c.**

- If **c <= N / c**, it means **c** is either the exact root or smaller than the root. We record **c** as our current best answer (**ans**) and narrow the search to the upper half **(a = c + 1).**
- If **c > N / c**, the value is too large, so we narrow the search to the lower half (**b = c - 1**).

  The process repeats until the lower bound crosses the upper bound (**a > b**).

```cpp
int bisection(int num) {
    if (num < 0) return 0;
    if (num == 0 || num == 1) return num;

    int a = 0;
    int b = num;
    int ans = 0;

    while (a <= b) {
        int c = a + (b - a) / 2;
        if (c <= num / c) {
            ans = c;
            a = c + 1;
        }
        else {
            b = c - 1;
        }
    }
    return ans;
}
```

(C++ Code)

Newton Method:

How it Works

Newton's method is an iterative numerical analysis technique used to find the roots of a real-valued function. To find **sqrt(N)** we solve for **f(x) = x^2 - N = 0**. Applying the standard Newton-Raphson formula **xn+1 = xn – (f(xn)/f'(xn))** yields the iterative sequence:

$$X_{n+1} = (X_n + (N/X_n))/2$$

When adapted to integer arithmetic, we apply the floor function to all divisions. The algorithm starts with an initial guess (commonly **x = N**). In

each loop, it computes a refined guess **(next_x).** Because of the quadratic convergence properties of Newton's method, the values rapidly decrease toward **sqrt(N)** The loop safely terminates the moment the sequence stops decreasing (**next_x >= x).**

```cpp
int newton(int num) {
    if (num < 0) return 0;
    if (num == 0 || num == 1) return num;
    int x0 = num;
    int x1 = (x0 + (num / x0)) / 2;
    while (x1 < x0) {
        x0 = x1;
        x1 = (x0 + num / x0) / 2;
    }
    return x0;
}
```

(C++ Code)

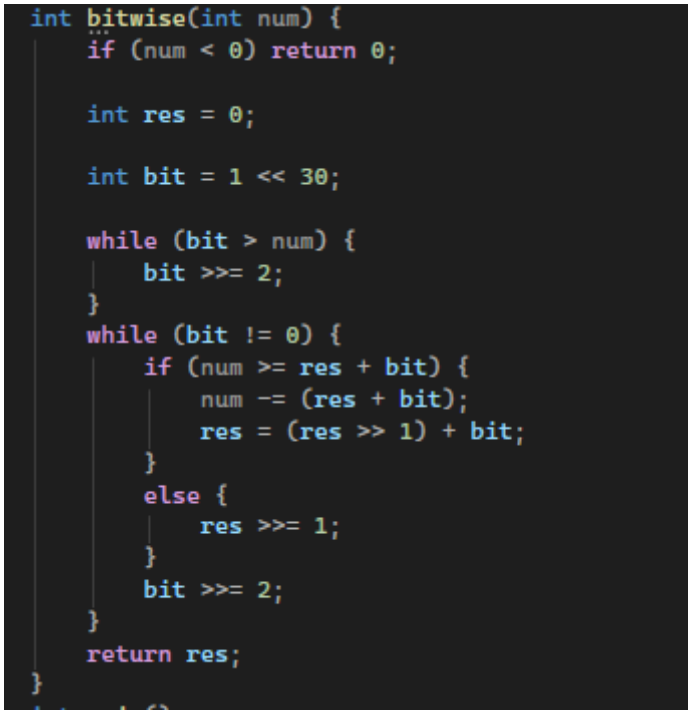
Bitwise Method:

How it Works

The Bitwise Digit-by-Digit method computes the square root bit-by-bit, from the most significant bit down to the least significant bit. This is the exact binary equivalent of the traditional long-hand manual square root method taught in schools.

Instead of processing base-10 digits, it scans the binary representation of the number in pairs of bits (since squaring a number doubles its bit length). It starts with a base bit set to the highest power of 4 (**2^30** for a 32-bit signed integer) and shifts down by 2 bits (**bit >>= 2**) in each step. At each iteration, it checks whether the remaining part of the input number **N** is greater than or equal to the current accumulated root plus

the active bit candidate (**res + bit**). If it is, that bit belongs to the final root; we subtract the value from **N** and update our result register.



```cpp
int bitwise(int num) {
    if (num < 0) return 0;

    int res = 0;

    int bit = 1 << 30;

    while (bit > num) {
        bit >>= 2;
    }
    while (bit != 0) {
        if (num >= res + bit) {
            num -= (res + bit);
            res = (res >> 1) + bit;
        }
        else {
            res >>= 1;
        }
        bit >>= 2;
    }
    return res;
}
```

(C++ Code)

Part 2:

1. BISECTION (BINARY SEARCH) METHOD

▪ Justification for Selection

I selected the **Bisection Method** because it is a highly intuitive and reliable numerical approach. It adapts the classic binary search concept to search for the integer square root within a bounded range [0, num]. It provides a rock-solid baseline because its behavior is completely predictable, and it guarantees finding the correct floor value of the square root without requiring complex calculus formulas.

▪ Efficiency Analysis

- **Number of Iterations:** Since the algorithm cuts the search space in half during every single step, its time complexity is strictly O(LOG2 N),where **N** is the input number. In terms of input size in bits (B), since N ≈ 2^B, the number of iterations scales linearly with the number of bits, executing at most B times (around 32 iterations for a full 32-bit integer).
- **Cost of Operations:** Inside the core loop, it uses 1 subtraction **(sub)** and 1 logical right shift **(srl)** to calculate the midpoint, which are very cheap (1 cycle each). However, it relies on a hardware division instruction (**div** and **mflo**) to check the condition **c <= num/c**. In MIPS architecture, a standard **div** instruction is expensive, typically consuming **30 to 40 clock cycles.**
- **Suitability for Assembly:** It is highly suitable because it completely avoids register overflow. In high-level languages, people often check **if (c * c <= num),** which easily overflows 32-bit registers for large numbers. In assembly, we can easily bypass this by restructuring the math into **num / c**, keeping the values safely within 32-bit limits.

2. NEWTON METHOD

▪ Justification for Selection

I selected the Newton Method because it is mathematically the fastest-converging algorithm among the three options. It uses an iterative calculus-based formula to refine guesses. For an engineering implementation, analyzing how a continuous mathematical function behaves when forced into discrete integer registers provides great insights into hardware-level truncation.

▪ **Efficiency Analysis**

- **Number of Iterations**: Newton's method boasts quadratic convergence, meaning the number of correct digits roughly doubles with each iteration. For a 32-bit input, it converges incredibly fast, usually requiring only **5 to 8 iterations** in the worst-case scenario. This makes its loop count significantly lower than both Bisection and Bitwise methods.
- **Cost of Operations**: While it runs very few iterations, the mathematical cost per iteration is heavy. Each loop requires 1 division (**div**), 1 addition (**add**), and 1 logical right shift (**srl**) to compute the average**: (x0 + num)/ 2**. Because the heavy **div** instruction (30-40 cycles) is executed in every single loop, the overall cycle count can still be high despite fewer iterations.
- Suitability for Assembly: This method is perfect for assembly because it leverages the hardware's dedicated **div** and **mflo** registers directly. Managing the loop termination condition**(x1 >= x0)** is incredibly straightforward using MIPS conditional branches (**slt** and **beq**), allowing us to stop the algorithm precisely at the integer floor without extra overhead.

3. BITWISE (DIGIT-BY-DIGIT) METHOD

▪ Justification for Selection

I selected the Bitwise Method because it represents the pinnacle of hardware-optimized programming. Instead of treating the input as an abstract mathematical value, it treats it as a raw sequence of binary digits, constructing the square root bit by bit from the most significant position down to the least. It is the most realistic representation of how a real processor's internal ALU calculates square roots.

▪ Efficiency Analysis

- **Number of Iterations:** The number of iterations is strictly deterministic and depends entirely on the bit-width of the architecture, not the magnitude of the input number. The first loop shifts the bitmask right by 2 bits to align it (max 15 times). The second loop runs exactly once for each remaining bit pair (15–16 times). In total, it executes a maximum of 30 **fixed iterations** regardless of whether the input is 5 or 2,000,000.
- **Cost of Operations:** This is the most efficient algorithm per iteration because it **completely eliminates division and multiplication**. It relies strictly on primitive, single-cycle hardware operations: logical right shifts (**srl**) to scale the mask/result, basic unsigned arithmetic (**subu**, **addu**), and standard branches (**bne**, **beq**). Every instruction inside the loop executes in exactly 1 clock cycle.
- **Suitability for Assembly:** It is exceptionally suited for assembly because it speaks the native language of the processor: bit manipulation. High-level languages abstract bit shifting away, but in MIPS assembly, operations like **bit >>= 2** map directly to a single machine instruction. It utilizes registers perfectly, runs lightning-fast, and guarantees zero risk of arithmetic overflow.

Part 3:

Bisection:

```
.text
.globl main

main:
    addiu $t2, $zero, 25

    jal func_bisection

    addu $a0, $v0, $zero
    addiu $v0, $zero, 1
    syscall

    addiu $v0, $zero, 10

    syscall
```

```
func_bisection:

slt $t1, $t2, $zero # t2 = num olsun
bne $t1, $zero, func_returnzero # IF (NUM < 0) RETURN 0;

beq $t2, $zero, func_returnitself
addiu $t8, $zero, 1
beq $t2, $t8, func_returnitself

add $t3, $zero, $zero # a = 0;
add $t4, $zero, $t2 # b = num;
add $t7, $zero, $zero # ans = 0;
j while_loop

while_loop:
slt $t1, $t4, $t3 # a > b
bne $t1, $zero, loop_end # while(!(a > b))
```

```
# c = a + (b-a) / 2;
add $t5, $zero, $zero # c = 0;

sub $t5, $t4, $t3 # b - a

srl $t5, $t5, 1 # (b-a)/2

add $t5, $t3, $t5 # c = a + (b-a) / 2;

# if(c <= num / c);
div $t2, $t5 # num/c
mflo $t6

slt $t1, $t6, $t5 # if (num/c < c)
bne $t1, $zero, if1_else
add $t7, $zero, $t5 # ans = c
addi $t3, $t5, 1 # a = c + 1
j while_loop

if1_else: # else{ b = c - 1;}
addiu $t4, $t5, -1
j while_loop

loop_end: # return ans
add $v0, $t7, $zero #return ans
jr $ra

func_returnzero: # return 0 nasıl yapılır ona bak
add $v0, $zero, $zero
jr $ra

func_returnitself: # return num(t2)
add $v0, $t2, $zero
jr $ra
```

Raw codes will be given at bottom of report

Edge Cases:

- **Negative Numbers:** If the input (num) is less than 0, it instantly jumps to func_returnzero and gives back 0.
- **0 and 1:** If num is 0 or 1, the square root is just the number itself. It catches this with beq and jumps straight to func_returnitself.

Initialization:

- $t3 is our lower bound (a = 0).
- $t4 is our upper bound (b = num).
- $t7 is our tracking variable (ans = 0), which will hold our best valid guess so far.

While_loop:

- **Loop Condition:** The loop runs as long as a <= b. The exact moment a > b (which slt checks), it breaks out and moves to loop_end.
- **Finding Midpoint (c):** To prevent any crazy overflow issues, it calculates the midpoint using the formula **c = a + (b-a)/2**. It subtracts **b - a**, uses **srl $t, 1** to quickly divide it by 2 (bit shifting is way faster than actual division), and adds it back to a. This midpoint c is stored in $t5.

If-Else Split:

- It runs div $t2, $t5 (**num/c**) and grabs the result from the hardware using **mflo $t6**.
- If **num/c < c** (**else1_else**): This means our guess **c** is too big. So, we shrink our upper bound by setting **b = c - 1** and jump back to the top.
- **Otherwise (If condition):** This means **c** is a solid candidate! We save it into our answer registry **(ans = c)** and try to see if there's a larger valid integer square root by bumping up our lower bound: **a = c + 1.**

Returns:

- **loop_end**: Loads the final saved answer (**$t7**) into the official return register **$v0** and uses **jr $ra** to bounce back to main.
- The edge case labels do the exact same thing, just loading a hardcoded **0** or the original num into **$v0** before exiting.

Newton Method:

```
.text
.globl main

main:
    addiu $t2, $zero, 25

    jal func_newton

    addu $a0, $v0, $zero
    addiu $v0, $zero, 1     # Ekrana integer yazdirma syscall'u
    syscall

    addiu $v0, $zero, 10    # Exit syscall'u

    syscall


func_newton:

slt $t1, $t2, $zero # t2 = num olsun
bne $t1, $zero, func_returnzero # IF (NUM < 0) RETURN 0;

# if (num == 0 || num == 1) return num;
beq $t2, $zero, func_returnitself
addiu $t8, $zero, 1
beq $t2, $t8, func_returnitself

add $t3, $zero, $t2 # x0 = num;
div $t2, $t3 # num/x0
mflo $t4

add $t4, $t4, $t3 # x0 + num/x0
srl $t4, $t4, 1 # x1 = (x0 + (num / x0)) / 2;
```

```
# x1 = $t4
# x0 = $t3
#while()
while_loop:
slt $t5, $t4, $t3 # x1 < x0
beq $t5, $zero, loop_end

add $t3, $zero, $t4 # x0 = x1;

div $t2, $t3 # num/x0
mflo $t6
add $t4, $t3, $t6
srl $t4, $t4, 1
j while_loop

loop_end:
add $v0, $t3, $zero #return x0
jr $ra

func_returnzero:
add $v0, $zero, $zero
jr $ra

func_returnitself: # return num(t2)
add $v0, $t2, $zero
jr $ra
```

Raw codes will be given at bottom of report

Edge Cases:

- **Negative Inputs:** It uses slt to check if num ($t2) is less than 0. If true, it immediately branches to func_returnzero.
- **0 and 1:** If num is exactly 0 or 1, the integer square root is identical to the input. The code catches these using beq and jumps straight to func_returnitself.

First Iteration:

Setting x0: It sets the initial guess ($t3, representing x0) equal to num itself.

Calculating: x1: x1 = (x0+(num/x0))/2

It divides num by **$t3,** adds **$t3** to the quotient, and shifts right by 1 bit (srl ..., 1) to divide by 2. This first calculated guess (x1) is stored in $t4.

While_loop:

- **The Condition:** It evaluates slt $t5, $t4, $t3, which checks if the new guess (x1) is strictly smaller than the previous guess (x0).
- **The Fixed Exit Point**: Because you updated it to **beq $t5, $zero, loop_end**, the loop safely breaks the exact moment the new guess stops getting smaller (i.e., when x1 >= x0). This indicates the algorithm has found the closest integer floor.

Updating the Guess Inside the Loop:

- **Shift Values:** The current guess becomes the old guess: x0 = x1 (add $t3, $zero, $t4).
- **Recalculate:** It performs the exact same division, addition, and right-shift sequence using the hardware's div and mflo to generate the next x1 value inside $t4.
- **Repeat:** It hits j while_loop to re-verify if the value can shrink any further.

Returning Result:

Once the loop terminates at loop_end, the most accurate approximation remains stored inside $t3 (x0). The code transfers this value to the official return register **$v0** and executes **jr $ra** to pass control back to your **main** block safely.

Bitwise Method:

```
.text
.globl main

main:
    addiu $t2, $zero, 25

    jal func_bitwise

    addu $a0, $v0, $zero
    addiu $v0, $zero, 1
    syscall

    addiu $v0, $zero, 10

    syscall
```

```
func_bitwise:

slt $t1, $t2, $zero # t2 = num olsun
bne $t1, $zero, func_returnzero # IF (NUM < 0) RETURN 0;

add $t1, $zero, $zero # res = 0

addiu $t0, $zero, 1
sll $t4, $t0, 30 # bit = 1 << 30;

# while( bit > num)
# while (bit != 0)
while2_loop:
beq $t4, $zero, loop2_end

# if(num >= res + bit) == if(!(num < res + bit))
addu $t6, $t1, $t4 # res + bit
slt $t5, $t2, $t6
bne $t5, $zero, else1

subu $t2, $t2, $t6 # num -= (res + bit);
srl $t1, $t1, 1 # res >> 1
addu $t1, $t1, $t4 # res = (res >> 1) + bit;

srl $t4, $t4, 2 # bit >>= 2;
j while2_loop

else1:
srl $t1, $t1, 1
srl $t4, $t4, 2 # bit >>= 2;
j while2_loop

loop2_end:
add $v0, $t1, $zero
jr $ra

func_returnzero:
add $v0, $zero, $zero
jr $ra
```

Raw codes will be given at bottom of report

Negative Check:

- **Negative Inputs:** It uses slt to check if num ($t2) is less than 0. If true, it immediately branches to func_returnzero.
- **0 and 1:** If num is exactly 0 or 1, the integer square root is identical to the input. The code catches these using beq and jumps straight to func_returnitself.

Initialization:

- **$t1** is cleared out to act as our result **(res = 0).**
- **$t4** is initialized as our bitmask. We take 1 and shift it left by 30 bits **(sll ..., 30)** to get **bit = 1 << 30**, which is our starting point.

while1_loop:

We can't start processing bits that are way bigger than our input number. So, this loop keeps shifting the mask 2 bits to the right **(srl ..., 2)** as long as **bit > num.** The moment the mask drops below or equals num, the loop ends and we move to the next step.

while2_loop:

- **The Check:** It calculates res + bit into $t6 and checks if our number is greater than or equal to this value.
- **The IF Path:** If num is bigger, it means this bit belongs in our square root. So, we subtract that value from num, update our res register (res = (res >> 1) + bit), shift the mask right by 2 bits, and loop back.
- **The ELSE Path:** If our guess is too big, we skip that bit. We just right-shift res by 1 bit, shift the mask right by 2 bits, and jump back to the top.

Returning Result:

Once the mask register **($t4)** completely drains down to 0, the loop breaks. The final answer is now sitting in **$t1**. We just copy it to the standard return register **$v0** and hit **jr $ra** to go back to main.

Part 4:

Input	Expected Result	Output(CODE)
0	0	0
1	1	1
2	1	1
3	1	1
4	2	2
9	3	3
16	4	4
25	5	5
198	?	14

1. Performance Comparison Table

To evaluate the trade-offs between the three implemented square root algorithms, we can summarize their core architectural and algorithmic behaviors in the following table:

Criteria	**Bisection Method**	**Newton-Raphson Method**	**Bitwise (Digit-by-Digit)**
Time Complexity	O(LOG N)	Quadratic Convergence	O(B) (Where B=32bits)
Average Iterations	Approx 25-32	Approx 5-8 Loops	Approx 30 loops
Heavy Instructions Used	div	div	None (Only shifts and adds)
Cycles per Iteration	High (Approx 40 - 50 cycles)	High(Approx 40-50 cycles)	Ultra Low
Hardware Efficiency	Medium	Medium	Excellent (ALU Friendly)

• **Summary**

In conclusion, while the Newton method provides an incredibly elegant mathematical shortcut with minimal iterations, the **Bitwise method is the most optimal and practical choice for low-level embedded systems and assembly computing**, as it speaks the native language of the processor's ALU: bit manipulation.

Raw Codes:

```
.text
.globl main

main:
    addiu $t2, $zero, 25

    jal func_bisection

    addu $a0, $v0, $zero
    addiu $v0, $zero, 1
    syscall

    addiu $v0, $zero, 10

    syscall

func_bisection:

slt $t1, $t2, $zero # t2 = num olsun
bne $t1, $zero, func_returnzero # IF (NUM < 0) RETURN 0;

beq $t2, $zero, func_returnitself
addiu $t8, $zero, 1
beq $t2, $t8, func_returnitself

add $t3, $zero, $zero # a = 0;
add $t4, $zero, $t2 # b = num;
add $t7, $zero, $zero # ans = 0;
j while_loop

while_loop:
slt $t1, $t4, $t3 # a > b
bne $t1, $zero, loop_end # while(!(a > b))

# c = a + (b-a) / 2;
add $t5, $zero, $zero # c = 0;

sub $t5, $t4, $t3 # b - a

srl $t5, $t5, 1 # (b-a)/2

add $t5, $t3, $t5 # c = a + (b-a) / 2;

# if(c <= num / c);
div $t2, $t5 # num/c
mflo $t6

slt $t1, $t6, $t5 # if (num/c < c)
bne $t1, $zero, if1_else
add $t7, $zero, $t5 # ans = c
addi $t3, $t5, 1 # a = c + 1
j while_loop

if1_else: # else{ b = c - 1;}
addiu $t4, $t5, -1
j while_loop

loop_end: # return ans
add $v0, $t7, $zero #return ans
jr $ra

func_returnzero: # return 0 nasıl yapılır ona bak
add $v0, $zero, $zero
jr $ra

func_returnitself: # return num(t2)
add $v0, $t2, $zero
jr $ra
```

```
.text
.globl main

main:
    addiu $t2, $zero, 25

    jal func_newton

    addu $a0, $v0, $zero
    addiu $v0, $zero, 1     # Ekrana integer yazdırma syscall'u
    syscall

    addiu $v0, $zero, 10    # Exit syscall'u

    syscall


func_newton:

slt $t1, $t2, $zero # t2 = num olsun
bne $t1, $zero, func_returnzero # IF (NUM < 0) RETURN 0;

# if (num == 0 || num == 1) return num;
beq $t2, $zero, func_returnitself
addiu $t8, $zero, 1
beq $t2, $t8, func_returnitself

add $t3, $zero, $t2 # x0 = num;
div $t2, $t3 # num/x0
mflo $t4

add $t4, $t4, $t3 # x0 + num/x0
srl $t4, $t4, 1 # x1 = (x0 + (num / x0)) / 2;

# x1 = $t4
# x0 = $t3
#while()
while_loop:
slt $t5, $t4, $t3 # x1 < x0
beq $t5, $zero, loop_end

add $t3, $zero, $t4 # x0 = x1;

div $t2, $t3 # num/x0
mflo $t6
add $t4, $t3, $t6
srl $t4, $t4, 1
j while_loop

loop_end:
add $v0, $t3, $zero #return x0
jr $ra

func_returnzero:
add $v0, $zero, $zero
jr $ra

func_returnitself: # return num(t2)
add $v0, $t2, $zero
jr $ra
```

```
.text
.globl main

main:
    addiu $t2, $zero, 198

    jal func_bitwise

    addu $a0, $v0, $zero
    addiu $v0, $zero, 1     # Ekrana integer yazdırma syscall'u
    syscall

    addiu $v0, $zero, 10    # Exit syscall'u

    syscall



func_bitwise:

slt $t1, $t2, $zero # t2 = num olsun
bne $t1, $zero, func_returnzero # IF (NUM < 0) RETURN 0;

add $t1, $zero, $zero # res = 0

addiu $t0, $zero, 1
sll $t4, $t0, 30 # bit = 1 << 30;

# while( bit > num)
while1_loop:
slt $t5, $t2, $t4
beq $t5, $zero, while2_loop # bit > num

srl $t4, $t4, 2 # bit >>= 2;  bit = bit >> 2;
j while1_loop

# bit = $t4
# res = $t1
# num = $t2

# while (bit != 0)
while2_loop:
beq $t4, $zero, loop2_end

# if(num >= res + bit) == if(!(num < res + bit))
addu $t6, $t1, $t4 # res + bit
slt $t5, $t2, $t6
bne $t5, $zero, else1

subu $t2, $t2, $t6 # num -= (res + bit);
srl $t1, $t1, 1 # res >> 1
addu $t1, $t1, $t4 # res = (res >> 1) + bit;

srl $t4, $t4, 2 # bit >>= 2;
j while2_loop

else1:
srl $t1, $t1, 1
srl $t4, $t4, 2 # bit >>= 2;
j while2_loop

loop2_end:
add $v0, $t1, $zero
jr $ra

func_returnzero:
add $v0, $zero, $zero
jr $ra
```

Ekin

Computer Engineering